

Chair of Computer Architecture (M. Sc.)

Model Enumeration using IPASIR-UP

Master's Thesis

First Examiner

Prof. Dr. Armin Biere

Second Examiner

Prof. Dr. xyz

Author

Timpe Horig

Matrikelnummer: 5108196

Geburtsort: Neustadt, Deutschland

Working Period

start date - end date

Abstract

Contents

1	Introduction	1
2	IPASIR	2
3	IPASIR-UP	3
4	CaDiCall	5
4.1	Conflict Driven Clause Learning	5
4.2	Chronological Backtracking	5
4.2.1	(i) Backtracking through book keeping	7
4.2.2	Backtracking after a model was found	7
4.2.3	Conflict induced backtracking	7
4.2.4	(ii) Backtracking through propagation	7
4.3	Implicant Shrinking	8
4.4	Deciding	11
5	Extending IPASIR-UP	11
5.1	Making IPASIR-UP the only interface needed	12
5.1.1	Backtracking after a model was found	12
5.1.2	Maintaining chronological backtracking	13
5.2	Implicant Shrinking	13
5.3	Optimization	13
6	Results	14
7	Conclusion	14

1 Introduction

The Boolean satisfiability problem (SAT) is defined as follows:

Given a propositional formula F over variables $V = \{v_1, \dots, v_n\}$, decide whether there exists an assignment $\alpha : V \rightarrow \{0, 1\}$ such that $\alpha \models F$.

This problem was the first to be proven NP-complete by Cook [1] in 1971 and independently by Levin [2] in 1973. As a result, finding efficient algorithms for the SAT problem quickly became one of the most important and influential challenges in computer science. Modern Conflict-Driven Clause Learning (CDCL) solvers are capable of handling formulas with millions of variables and clauses. As a result, SAT solving plays a central role in hardware and software verification, automated reasoning, planning, scheduling, model checking, and cryptanalysis.

State-of-the-art solvers such as CaDiCal, Glucose, and Kissat demonstrate how theoretical advances combined with low level optimizations can transform an intractable worst-case problem into a powerful practical tool. These solvers typically focus on the decision problem: determining whether at least one satisfying assignment exists. If the formula is satisfiable, the solver returns a single model.

However, many applications require more than just one solution. In areas such as configuration, diagnosis, quantitative reasoning, explanation generation, and formal verification, it is often necessary to enumerate all satisfying assignments or a specific subset of them. This task, known as model enumeration (or All-SAT), generalizes the classical SAT problem from computing a single satisfying assignment to computing the whole set $\{\alpha : V \rightarrow \{0, 1\} \mid \alpha \models F\}$. While conceptually straightforward, model enumeration introduces significant algorithmic challenges. The number of satisfying assignments may be exponential in the number of variables, and naive approaches can quickly become infeasible.

A common technique for model enumeration is the iterative use of a SAT solver combined with blocking clauses. After each model is found, a clause composed of the negation of every literal in that model is added to the CNF formula, ensuring that the solver does not rediscover the same assignment. With the introduction of standardized interfaces such as IPASIR, SAT solvers can be seamlessly integrated into larger systems without the need to maintain the solver itself. More recently, IPASIR-UP was introduced, expanding IPASIR with User Propagators. This allows adding clauses without restarting the search. Using IPASIR-UP with Blocking Clauses yields a very clean model enumerator, that benefits from the underlying solver CaDiCal while CaDiCal itself remains unchanged. Although effective, this approach has several drawbacks. The repeated addition of blocking clauses increases the size of the formula, thereby degrading solver performance by affecting propagation efficiency and memory consumption. As the number of models grows, these effects scale up, eventually making blocking clauses infeasible.

In 2024 [3] presented TabularAllSat: A Sat solver that is especially designed for

disjoint projected enumeration for SAT and SMT without blocking clauses. TabularAllSat outperforms all other state-of-the-art AllSAT solvers. This was achieved by "combining the best of current All- SAT state-of-the-art literature": (i) Conflict Driven Clause Learning short CDCL, (ii) Chronological backtracking to avoid blocking clauses and (iii) Implicant shrinking to minimize models.

Besides adding clauses iteratively, IPASIR-UP also allows to interfere with decision making, backtracking and found models. This raises the question whether it is possible to create a model enumerator through IPASIR-UP without blocking clauses by using all state-of-the-art techniques from TabularAllSat.

This thesis introduces CaDiCaL, a model enumerator which is almost entirely implemented through the IPASIR-UP interface and does not need any blocking clauses. "Almost" because there are two minor additions that need to be added to IPASIR-UP: One is already added to the newest version of CaDiCaL's IPASIR-UP interface and the other is fixing a bug introduced by the former. As IPASIR-UP is an interface for CaDiCaL and CaDiCaL being a CDCL-based solver, technique (i) from TabularAllSat is naturally covered. Also technique (ii) chronological backtracking is fully possible through the IPASIR-UP interface and thus also used in CaDiCaL. Using (iii) the implicant shrinking algorithm proposed by TabularAllSat requires access to the watch-lists of literals. This is not supported by the IPASIR-UP interface. However, it is still implemented in CaDiCaL and section ... discusses which additional functionality is needed and how it could be added to the IPASIR-UP interface.

The model enumerator supports a fixed variable order. However, per default CaDiCaL decides. It also can be used as a simple model counter, which makes it possible to count models for formulas with exponential many models while keeping memory consumption very little. Furthermore, as IPASIR-UP also introduces the possibility to propagate literals. It is also possible to backtrack through adding reason clauses if necessary.

2 IPASIR

The IPASIR interface standardizes incremental interaction with CDCL-based SAT solvers by defining a minimal set of functions through which the user can call the solver in bigger contexts while keeping the program Independent of the solver. A solver instance is created and subsequently manipulated only via functions such as `ipasir_add` to add literals to the current clause, terminated by 0. `ipasir_assume` to add temporary assumption literals for the next call to `ipasir_solve`, and `ipasir_solve` to start or resume the search. After `ipasir_solve` returns SAT or UNSAT, the user may query models with `ipasir_val` or inspect a failed core via `ipasir_failed`. The interface deliberately abstracts from internal solver mechanisms: There is no access

to decision levels, propagation events, learned clauses, or backtracking. Although learned clauses are preserved across calls in incremental mode, the user cannot intervene during the search itself. Consequently, model enumeration requires blocking clauses: After finding a model, add its negation as a new clause to the cnf (via `ipasir_add`) and restart the solver. If the solver returns UNSAT, all models are found. However, this approach is sub-optimal as each new blocking clause increases the cnf. This makes it infeasible for big CNFs with many models. Model enumeration without blocking clauses through IPASIR is impossible since the solver terminates immediately upon finding a satisfying assignment.

3 IPASIR-UP

To overcome the limitations from IPASIR, Katalin Fazekas et al. introduced IPASIR-UP in 2023 as an extension of IPASIR that introduces **user propagators** (UP). IPASIR-UP augments the classical interface with a bidirectional communication layer between the solver’s CDCL loop and user-defined callbacks. Instead of restricting interaction to clause addition and top-level solve calls, IPASIR-UP enables controlled intervention during propagation, decision making, backtracking, and model detection.

- Model validation

```
bool cb_check_found_model(const tmodel &model) override {
    return true;
}
```

This function is called whenever the solver derives a total satisfying assignment. If the user propagator returns `true`, the solver reports SAT as usual. If it returns `false`, the model is rejected and the solver continues the search without restarting. Crucially, the internal trail, learned clauses, and heuristic state remain intact.

- External clause injection

```
bool cb_has_external_clause(bool &is_forgettable) override {
    return false;
}

int cb_add_external_clause_lit() override { return 0; }
```

These functions allow the user propagator to provide clauses to the solver right after `cb_check_found_model` was called. When `cb_has_external_clause` returns `true`, the solver repeatedly calls `cb_add_external_clause_lit` to obtain the literals of the clause (terminated by 0).

- Notifications

```
void notify_assignment(const ivec &list) override {}  
void notify_new_decision_level() override {}  
void notify_backtrack(size_t new_level) override {}
```

These notification callbacks inform the user propagator about changes to the assignment trail. `notify_assignment` communicates newly assigned literals, `notify_new_decision_level` signals that a decision was made and thus the decision level increased, and `notify_backtrack` reports to which level the solver backtracked.

- Decision making

```
int cb_decide() override { return 0; }
```

This callback is invoked before the solver makes the next decision. If it returns a non-zero literal, the solver uses it as the next decision. If 0 is returned, the solver makes its own decision, based on its internal decision heuristic.

- User propagation

```
int cb_propagate() override { return 0; }  
int cb_add_reason_clause_lit(int propagated_lit) override {  
    return 0;  
}
```

`cb_propagate` enables the user propagator to return implied literals under the current partial assignment. If a conflict is detected in which the propagated literal was involved, the solver asks for a reason clause through repeatedly calling `cb_add_reason_clause_lit` (terminated by 0). This enables steering through the decision space while adding the reason clauses only if really necessary.

A user must define a User Propagator defining all the functions above.

```
void connect_external_propagator (ExternalPropagator* propagator);  
void disconnect_external_propagator ();  
void add_observed_var (int var);  
void remove_observed_var (int var);
```

Then User Propagator can simply be connected and disconnected to the solver by calling `connect_external_propagator` and `disconnect_external_propagator`.

The notification callback functions are only called for variables marked as `observed`. The user can mark and unmark variables through the functions `add_observed_var` and `remove_observed_var`.

4 CaDiCAll

CaDiCAll is a fully functional model enumerator using CaDiCaL almost exclusively through the IPASIR-UP interface. It is implemented by using all three state-of-the-art mechanisms defined by Spallitta et al[3]: (i) Conflict Driven Clause Learning (Section 4.1), (ii) Chronological Backtracking (Section 4.2) and (iii) Implicant Shrinking (Section 4.3).

4.1 Conflict Driven Clause Learning

Conflict-Driven Clause Learning (CDCL) improves the classical DPLL algorithm by learning from conflicts. When the solver runs into a conflict, it does not simply flip the last decision but analyses the conflict to determine the assignments that caused the contradiction.

The analysis starts with the conflicting clause (the clause that is falsified by the current assignment). The solver inspects which literals in that conflict clause were forced by another clause and resolves them with each other. Let the conflict clause be $(A \vee \neg x_1)$ and x_1 be forced by $(B \vee x_1)$. Resolving these two clauses on x_1 yields the reason clause $(A \vee B)$. This process is repeated until a learned clause of the form $(A \vee B \vee \dots)$ is obtained that contains exactly one literal from the current decision level, which is then added to the CNF. Such a clause enables non-chronological backtracking: the solver backtracks to the second highest decision level occurring in the clause. At that point, all other literals in the clause remain falsified while the literal from the previous decision level becomes the only unassigned one, making the clause unit and forcing a new assignment through unit propagation. This prevents the solver from encountering the same conflict again.

By learning new clauses and pruning conflicting regions of the search space, CDCL significantly improves the efficiency of SAT solving. For this reason, it forms the basis of most modern SAT solvers.

As CaDiCaL is a CDCL-based solver, CDCL is implicitly provided to CaDiCAll when interfacing with CaDiCaL via IPASIR-UP.

4.2 Chronological Backtracking

Modern SAT solvers based on Conflict-Driven Clause Learning (CDCL) employ non-chronological backtracking (NCB). During conflict analysis, the solver may determine that some earlier decisions did not contribute to the conflict. As a result,

it can backtrack multiple decision levels at once, effectively skipping parts of the search tree that cannot lead to a solution. While this greatly improves efficiency by pruning large portions of the search space, it also allows the solver to alter the order in which decisions are revisited.

In contrast, chronological backtracking (CB) always undoes only the most recent decision. The solver then either assigns the opposite value to that variable or proceeds with further propagation. Since decisions are revisited strictly in reverse order, the structure of the search tree remains fixed, which is why this approach is referred to as chronological backtracking.

When enumerating all models, it is essential to ensure that no model is reported more than once. Conceptually, this requires keeping track of which regions of the search space have already been explored. However, non-chronological backtracking can reshape the search tree dynamically, making the exploration order less predictable. Consequently, once a model is found, it is necessary to prevent the solver from rediscovering the same assignment. Since NCB may jump back to an arbitrary earlier decision level, all decisions that led to the model must be considered when excluding that path, ensuring that the solver cannot revisit the exact same assignment again.

(TODO: why NCB and not CB)

Chronological backtracking can be set as an option in CaDiCaL. Thus, backtracking as a result of running into a conflict is handled by the solver itself. However, after a model is found, the solver must also backtrack in order to continue the search for different models in other branches of the search space. This makes backtracking non trivial, as the paths to the found models must be excluded from future exploration. This can be achieved through two different methods: (i) When a model is found, undo the last decision and decide its negation. (ii) When a model is found, undo the last decision and propagate its negation. While the later does not require book keeping of the decisions themselves, it may require a reason clause for that propagation if it is involved in another backtracking. Hence introducing, even if smaller and not always, blocking clauses. Thus, this thesis focuses on the former method and fully avoiding adding clauses. Still, the later method is available in CaDiCaL as a command line option (`-reason`) and also part of the benchmarks in section X.

Backtracking after a model is found was not supported by the IPASIR-UP interface. However, as discussed in Section X, allowing to backtrack after a model is found does not introduce any bad behavior and thus was added to the newest version of CaDiCaL’s IPASIR-UP interface by Katalin Fazekas.

4.2.1 (i) Backtracking through book keeping

In order to know which decision was made on which decision level, two information are relevant: (i) the phase of the decided variable and (ii) how often this variable has been decided. A count of 1 meaning, this decision has been made exactly once. A count of two indicating that this decision already has been flipped. Thus, finding the right decision level to backtrack to, simply becomes finding the highest decision level with a variable count of 1.

4.2.2 Backtracking after a model was found

After a model is found, either there is no decision with a decision count 1 and thus, all paths have been discovered. Then, the solver can terminate. Otherwise, there exists a decision level with a decision count of 1 that the solver can backtrack to. Let that level be l . Then the solver must backtrack to the decision level $l - 1$. However, before `cb_decide` is called and the decision can be flipped, the solver propagates. This can lead to four different outcomes: (i) The solver only propagates variables that are different to the decision or none at all. In that case in `cd_decide` the decision can be flipped and returned. (ii) The solver propagates the negation of the decision itself. Then the solver can choose the next decision itself, `cb_decide` returns 0. (iii) While propagating the solver runs into a conflict and backtracks again. This case is equivalent to a normal backtrack. (iv) Due to propagation of literals the old decision itself is implied again. The solver starts to rediscover the old path. This is detected directly in `notify_assignment` but can be handled the earliest when either `cb_decide` or `cb_check_found_model` is called. Then the solver is forced to backtrack further.

4.2.3 Conflict induced backtracking

If a conflict occurs the solver automatically backtracks one decision level. However, as the solver does this on its own, it backtracks to the last decision, no matter what decision count it has. If the decision count is 1, the same four outcomes as above can occur. However, if the decision count is 2 the solver wants to flip the decision again. From this moment on, everything the solver does, must be undone again. Again this is detected and handled by the user propagator. Unfortunately, this can lead to an overhead as the solver may spend some time propagating.

4.2.4 (ii) Backtracking through propagation

An other approach is not to decide the negation of the last made decision but to propagate it. Thus, if the solver backtrack again, it simply backtracks over it, as it is not a decision anymore. However, if the propagated literal is inspected while conflict analysis, the solver needs a reason clause for it. The reason clause for this

literal is just the negation of all decisions made on a lower level and thus can be constructed on the fly and does not need to be stored. This fits perfect with the design of IPASIR-UP as it asks for reason clauses only if necessary. Thus, adding reason clauses has two advantages to adding blocking clauses: (i) the size can be smaller as it only contains decisions on a lower level. (ii) they are only added if necessary.

4.3 Implicant Shrinking

Models (also called implicants) found by the solver are total. Models are total if they contain every variable. For a CNF $(x_1 \vee x_2) \wedge (x_1 \vee \neg x_2)$ the model enumerator returns two total models: $x_1 \wedge x_2$ and $x_1 \wedge \neg x_2$. As x_1 occurs in both models positively, the only difference lays in x_2 which occurs once positively and once negatively. In other words, if x_1 is positiv, it does not matter what the value of x_2 is, it is always a model for the CNF. Thus, these models can be merged into one model by dropping x_2 , resulting in one partial model x_1 . This reduces both, the amount of models in total and the size of models. It also may fasten the solving itself. Normally, after finding the first model $x_1 \wedge x_2$ the model enumerator backtracks to the last decision that can be flipped. Assuming the decisions where x_1 at decision level 1 and x_2 at decision level 2, the model enumerator would backtrack to decision level one and then continue with $\neg x_2$. However, if the model enumerator shrinks the model $x_1 \wedge x_2$ to x_1 the last decision that can be flipped is x_1 . Thus, the model enumerator backtracks to decision level 0 and can continue with $\neg x_1$, skipping the branch with $\neg x_2$ completely. As the branch containing x_2 could be arbitrary large, this may reduce the search space significantly.

Finding out which literals can be dropped is non trivial. Implicant shrinking algorithms from non chronological backtracking based model enumerators do not guarantee mutual exclusiveness between different assignments[3]. Spallitta et al. [3] introduces their own shrinking algorithm that is (i) compatible with chronological backtracking and (ii) tries to cut the highest amount of literals while still ensuring mutual exclusivity. This algorithm starts with the last literal on the Trail and tries to drop as many literals as possible. A literal ℓ can be dropped if ℓ is a decision and all clauses in which ℓ occurs contain another literal ℓ' that satisfies the clause.

In CaDiCaL the 2-watched-literal scheme is used. This allows the solver not to constantly check all clauses if the assignment of a literal changes. Instead every clause of the CNF is only watched by two literals, let them be ℓ_1 and ℓ_2 . This is enough to track the status of the clause if the following invariant holds: At least one literal is not falsified. In order to keep this invariant alive, if one watched literal ℓ_1 becomes falsified the solver tries to find a replacement. If there is another literal ℓ' in that clause that is neither false nor ℓ_2 , it replaces ℓ_1 as the watching literal. If there exists no such literal ℓ' the solver knows that ℓ_2 is the only literal left that could

satisfy the clause and forces ℓ_2 . If both literals ℓ_1 and ℓ_2 are falsified, the clause is falsified, yielding UNSAT.

If we want to drop a literal ℓ , it suffices to check all clauses that are watched by ℓ instead of all clauses in which ℓ occurs. If a model is found, we now at least one of the watching literals for every clauses is satisfied. Thus, dropping a literal that is not watched will never falsify a clause. However, if we drop a literal ℓ that is currently watching a clause c , we know there exists another literal $\ell' \in c$ that satisfies c . This literal ℓ' must now become the new watching literal for c . Otherwise, when trying to drop ℓ' , c would not be checked.

Algorithm 1 IMPLICANT-SHRINKING(T)

```

1:  $b \leftarrow 0$ 
2:  $T' \leftarrow T$ 
3: while  $T' \neq \epsilon$  do
4:    $\ell \leftarrow T'.\text{pop}()$ 
5:   if  $\rho(\ell) \neq \text{DECISION}$  then
6:      $b \leftarrow \max(b, \delta(\ell))$ 
7:   else if  $\delta(\ell) > b$  then
8:      $b \leftarrow \text{CHECK-LITERAL}(\ell, b, T')$ 
9:   else if  $\delta(\ell) = 0$  or  $(\delta(\ell) = b \text{ and } \rho(\ell) = \text{DECISION})$  then
10:    break
11:   end if
12: end while
13: return  $b$ 

```

Going through all literals in all clauses watched by ℓ may introduce a bottleneck. Thus Spallitta et al [3] introduced a faster but more conservative algorithm exploiting the 2-watched-literal scheme. Conservative meaning it may not drop all literals that could be dropped.

The 2-watched-literal scheme is used in the the faster approach proposed from Spallitta et al. [3] as shown in algorithm 2.

Algorithm 2 CHECK-LITERAL(ℓ, b, T') (faster but more conservative)

```

1: for all  $c \in \omega(\ell)$  do
2:   if  $\exists \ell'$  such that  $c \in \omega(\ell')$  and  $\ell' \neq \ell$  and  $\ell' \in T'$  then
3:     Continue
4:   else
5:      $b \leftarrow \max(b, \delta(\ell))$ 
6:   end if
7: end for
8: return  $b$ 

```

Instead of checking all other literals in the watched clause, simply check the other literal ℓ' that watches that clause. If it satisfies the clause as well, the literal ℓ can be dropped. As ℓ' already watches the clause, nothing must be updated. This is more conservative as it may not drop literals that could be dropped. If ℓ' is falsified

but another literal in that clause is satisfied, ℓ still could be dropped but is not. However, as this conservative approach does not need to traverse the clause and check each literal, it is significantly faster.

As IPASIR-UP does not allow any access to watches, this is implemented by using a backdoor to CaDiCaL directly. Section XX discussed what is needed and how it could be added to the IPASIR-UP interface.

Adapting this implicant shrinking algorithm to the model enumerator using IPASIR-UP may introduce a fault. As stated in Section 5.1 and 5.2 there are two different ways to flip a decision. If the decision is flipped by deciding its negation, algorithm 1 must be adapted.

Consider the CNF $(x_1 \vee x_2 \vee x_3)$. Let x_2 and x_3 be the watched literals at the start and the decision order be $x_1 \rightarrow x_2 \rightarrow x_3$. The first model to be found will be $x_1 \wedge x_2 \wedge x_3$. Implicant shrinking will first remove x_3 as the other watched literal (x_2) still satisfies the clause. Then it will try to remove x_2 . However, as the other watched literal is x_3 which is no longer on the trail, it cannot continue. Thus, discovering the model $x_1 \wedge x_2$. Then the solver backtracks and flips x_2 by deciding its negation $\neg x_2$. Remind that $\neg x_2$ is still a decision. As x_2 is one of the watched literals but is now falsified, x_1 becomes the new watched literal, as it is the only literal in the clause that is neither false nor already watched. Then the solver decides x_3 resulting in the model $x_1 \wedge \neg x_2 \wedge x_3$. Now implicant shrinking starts again. First removing x_3 as the other watched literal (x_1) still satisfied the clause. Next it tries to remove $\neg x_2$ as it is the next highest decision. As $\neg x_2$ does not occur in any clauses, it cannot be a watched literal. Thus removing it does not falsify any clause. Then, it tries to remove x_1 . However, as the other watched literal is x_3 which was already removed, x_1 can not be removed. Hence, yielding the model x_1 .

The implicant shrinking algorithm yielded both models $x_1 \wedge x_2$ and x_1 . These models are not distinct, as setting x_2 in the later generates the former.

This can be fixed by allowing implicant shrinking only to remove literals with a decision count of 1 (line 5 in algorithm 3). Consider a variable with decision count 2 to be checked by implicant shrinking. There are two possibilities: (i) The variable could not be removed when it's decision count was 1. Removing it now could trigger exactly a case like the example above. (ii) The variable was removed when it's decision count was 1. In that case the model enumerator would have backtracked over the variable and thus resetting the decision count, which makes this case impossible to reach.

Flipping decisions by propagation their negations (Section 5.2) is not affected by this fix, as the highest decision count is always one.

Algorithm 3 IMPLICANT-SHRINKING(T)

```
1:  $b \leftarrow 0$ 
2:  $T' \leftarrow T$ 
3: while  $T' \neq \epsilon$  do
4:    $\ell \leftarrow T'.\text{pop}()$ 
5:   if  $\rho(\ell) \neq \text{DECISION}$  and  $\text{decision\_count}(\ell) = 1$  then
6:      $b \leftarrow \max(b, \delta(\ell))$ 
7:   else if  $\delta(\ell) > b$  then
8:      $b \leftarrow \text{CHECK-LITERAL}(\ell, b, T')$ 
9:   else if  $\delta(\ell) = 0$  or  $(\delta(\ell) = b \text{ and } \rho(\ell) = \text{DECISION})$  then
10:    break
11:   end if
12: end while
13: return  $b$ 
```

4.4 Deciding

The order in which decisions are made can heavily impact the performance of sat solvers. The right decision can lead the solver into early conflicts, pruning big parts of the search space completely or skipping long nested parts. While in model enumeration the later benefit is lost as all models must be found, it still profits from early conflicts.

In order to search systematically through the whole search tree, a fixed variable order is the easiest. On each `cb_decide` call, the next undecided variable is returned, assigning this variable positiv. Backtracking also becomes trivial, always backtrack to the last positive decision. However, this constraint can be weakened while still preserving a systematically order. First, the phase of a decided variable can be chosen freely, while keeping backtracking still simple enough. Instead of backtracking to the last positive decision, simply backtrack to the last decision that has a decision count of 1. Meaning it has been decided exactly once. Second, the order in which variables are decided can also be chosen complete free, as long as after a backtracking the decision is just flipped. This actually becomes non trivial as discussed in the section Chronological Backtracking. However, enables the solver to profit from dynamic decision heuristics such as Variable State Independent Decaying Sum (VISIDS) which is known to improve solver performance significantly and thus is a commonly used decision heuristic in modern sat solvers. The performance difference between fixed and dynamic decision orders can be seen in

5 Extending IPASIR-UP

We propose three kinds of extensions to IPASIR-UP: (i) Additional features required to make model enumeration using IPASIR-UP possible. (ii) Additional features to allow implicant shrinking and (iii) Additional features to avoid duplications and thus

making model enumeration faster using IPASIR-UP.

5.1 Making IPASIR-UP the only interface needed

5.1.1 Backtracking after a model was found

In conventional SAT solving, backtracking only occurs when the solver encounters a conflict. To continue the search, the solver must undo at least one previous decision and explore a different branch of the search space.

However, a similar situation arises when enumerating all satisfying assignments (models). Once the solver finds a model, the current assignment satisfies all clauses and therefore no conflict is present to trigger backtracking. Nevertheless, the search must continue in order to determine whether additional models exist. Thus, the solver must be forced to leave the current branch of the search space and explore alternative assignments.

When a model is found, IPASIR-UP allows the user propagator to observe and reject the model by returning `false` in `cb_check_found_model`. It then calls `cb_has_external_clause` if the user propagator returns `true`, the solver repeatedly calls `cb_add_external_clause_lit` allowing the user propagator to add the negation of the model as a blocking clause. This forces the solver to backtrack and explore other branches of the search space. However, as we do not want to add blocking clauses, CaDiCaL always returns `false` in `cb_check_found_model`. Which means, the solver is not forced to backtrack and would report the same model directly again. Resulting in an infinite loop where the solver is stuck.

In order to allow the user propagator to maintain additional constraints, IPASIR-UP provides the function `force_backtrack`. This function is allowed to be called inside `cb_decide`. It enables the user propagator to force the solver to backtrack to a specific decision level before making a new decision. Forcing the solver to backtrack inside `cb_decide` does not introduce bad behavior as at this point, the trail is fully consistent: All assignments up to the current decision level do not falsify the CNF, they are not in conflict with any other propagations and no propagations are pending.

When calling `cb_check_found_model` the trail is in an equally consistent state as in `cb_decide`: The trail fully satisfies the CNF and as it is a total assignment, there are no pending propagations. Thus, allowing `force_backtrack` in `cb_check_found_model` would not introduce any bad behavior.

To allow `force_backtrack` to be called from within `cb_check_found_model` simply the flag `forced_backt_allowed` must be set to `true` in the solver before calling `cb_check_found_model`.

This feature was added to the newest version of CaDiCaL's IPASIR-UP interface by Katalin Fazekas. Thus, allowing CaDiCaL to backtrack to the last decision after

a model is found and continuing the search from there without adding blocking clauses.

5.1.2 Maintaining chronological backtracking

When the solver is about to make the next decision, the solver internally calls `ask_decision`, which then calls `cb_decide`. The user propagator is allowed to call `force_backtrack` inside `cb_decide`. Backtracking might make the decision returned by the user propagator invalid, as it might be already assigned due to propagations after backtracking. However, instead of asking the user propagator again, the solver decides then on its own. This imposes a problem as CaDiCaL does not allow all decision to be made by the solver, some decisions are already fixed in order to force the solver to explore the search space chronologically. This can be fixed by calling `ask_decision` recursively instead of letting the solver decide. This way, the user propagator can make a new decision after backtracking and thus keep the exploration of the search space chronological.

If the user propagator backtracks below the pseudo decision level of the solver. The solver must go back to the CDCL loop instead of making a decision. In that case the solver ignores the decision made by the user propagator, continues with the CDCL loop and calls `ask_decision` again when it is about to make the next decision. However, the CDCL loop may propagate literals that lead to paths that have already been explored. Thus, the user propagator must check whether the work done by the CDCL loop has led to such a path and if so, backtrack further. If not, the user propagator can either return the original decision or make a new one if the CDCL loop already propagated it.

5.2 Implicant Shrinking

The implicant shrinking algorithm described in section X needs access to the all watch-lists of a given literal. However, the implicant shrinking algorithm uses them read only. Thus, exposing a read only reference to them is unproblematic and could be integrated into IPASIR-UP without any changes to CaDiCaL itself. Here it is important to note that literals can be named differently external and internal. However, there already exist functions to translate from and to and thus the internal representation can be completely hidden through the IPASIR-UP interface.

5.3 Optimization

Although the IPASIR-UP interface provides enough information to build an external copy of variable assignments and stack state, maintaining them takes in average XX percent of the total running time of this model enumerator. Allowing read

only access to CaDiCaLs variable assignment and stack would make all external representation redundant.

6 Results

...

7 Conclusion

...

References

- [1] Stephen A. Cook. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.
- [2] Leonid A. Levin. Universal search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.
- [3] Giuseppe Spallitta, Roberto Sebastiani, and Armin Biere. Disjoint partial enumeration without blocking clauses. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(8):8126–8135, Mar. 2024. doi: 10.1609/aaai.v38i8.28652. URL <https://ojs.aaai.org/index.php/AAAI/article/view/28652>.

Erklärung

Hiermit versichere ich, die vorliegende Arbeit ohne unerlaubte Hilfe und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt zu haben. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen sind, habe ich als solche kenntlich gemacht. Die eingereichte Masterarbeit wurde weder vollständig noch in wesentlichen Teilen Gegenstand eines anderen Prüfungsverfahrens. Die elektronische Version der eingereichten Masterarbeit stimmt in Inhalt und Formatierung mit den auf Papier ausgedruckten Exemplaren überein.

Freiburg im Breisgau, den 01.01.2016.